RESILIENCE GRAPH AI

Behavioural detection, attack path reconstruction and guided response for critical national infrastructure

Problem Statement 7

AI Driven Cyber Resilience for Critical National Infrastructure

ET AI Hackathon 2026

Team rishiikumarsingh2201

Team members	Rishii Kumar Singh, Sarthak Tomar, Aman Kumar, Aarushi Aanand

Project links

Resource	Link
Live application (Render)	https://resilience-graph-ai.onrender.com
Source code (GitHub)	https://github.com/RishiiGamer2201/resilience-graph-ai
Presentation (Canva)	https://canva.link/f4gesmsduelihuz
Submission document	https://docs.google.com/document/d/10fs9PBoaEQLUJkyoJPSqgNp-ejRsWS90C37CWV8kDgU/edit
Dataset	https://www.kaggle.com/datasets/c3c7d72d2098d35857c2136a6d1c35785b7ba94e0f48ed6de68d0ab1ed021945
Competition	https://unstop.com/competitions/crp-et-ai-hackathon-20-economic-times-1675680

A note on the numbers in this document: every figure is measured, and each one traces to an evaluation report or a measurement file in the repository. Where a number comes from outside our own work it is attributed to its source. Where a result is weak, or where a component is simulated rather than live, this document says so directly.

Contents

1. Executive summary

An attacker breaks into a hospital network. They do not break anything. They steal one employee's password and quietly log into machine after machine, looking for the patient database. Every one of those logins looks completely normal on its own. That is why intrusions of this kind go undetected for a global median of about ten days.

Resilience Graph AI reads the ordinary authentication logs an organisation already collects, and finds the story hidden across them. It does not look for known bad software. It learns what normal behaviour looks like for each account, then measures deviation from it.

On a real, labelled red team campaign the system produces the following, live, in under one fifth of a second:

What the system does	Measured result
Scores every authentication event and flags the anomalies	2,732 events scored, 1,192 flagged
Collapses those alerts into a single narrated incident	1,192 alerts become 1 incident
Reconstructs the attacker's movement across the estate	479 machines, 502 movements, 4 attacker controlled hosts
Identifies the single best machine to isolate first	Isolating 1 host severs 463 machines of exposure
Detects the attack without ever seeing an attack label in training	ROC-AUC 0.988 against 702 real red team events

The system additionally maps every step to the MITRE ATT&CK catalogue, the industry standard naming scheme for attacker techniques, predicts the likely next technique with a real transition probability, ranks which known threat group the behaviour resembles with a written justification, and recommends containment that a human must approve before anything happens.

Three properties separate this from a demonstration. First, the whole pipeline runs live on any log submitted to it, including a file uploaded by a judge. Second, no number displayed anywhere in the application is hard coded; each one is computed by the analysis that is currently loaded. Third, the parts that are simulated, specifically the containment actions, are labelled as simulated everywhere they appear.

2. The problem

2.1 The Indian context

Fact	Figure	Source
Cyber security incidents handled by CERT-In during 2023	1.59 million and above	CERT-In
Indian government entities operating end of life IT	Over 70 percent	PS-7 problem brief
Global median attacker dwell time	About 10 days	Mandiant M-Trends 2024

Two Indian precedents shaped this project directly. The ransomware attack on AIIMS Delhi in 2022 took the country's premier hospital offline for days. The breaches at the Central Board of Secondary Education affected the national school examination system. In both categories of organisation the defining constraint is the same: critical systems, legacy infrastructure, and no capacity to staff a round the clock security operations centre.

2.2 Why existing defences miss this class of attack

Defence	How it works	Why it misses a quiet credential based intrusion
Antivirus and signatures	Matches known bad files and patterns	The attacker deploys no malware. They use valid logins.
Firewall	Blocks unauthorised network connections	The attacker is already inside, using permitted paths.
Threshold rules, for example alert on five failed logins	A limit on a single counter	A patient attacker simply stays under the threshold.
Standard SIEM alerting	Scores each event independently	One intrusion becomes thousands of disconnected alerts, and the pattern across them is never assembled.

2.3 The three failures we set out to fix

- **Signature evasion.** An attack carried out with stolen but valid credentials matches no known bad rule, because nothing about it is technically unauthorised.
- **Alert fatigue.** Because each event is scored alone, a single intrusion is presented to an analyst as thousands of separate rows. Analysts triage rows, miss the pattern, and burn out.
- **No blast radius view.** Even when one alert is investigated properly, nobody can see the path from a compromised workstation to the patient database, or answer the only question that matters during an incident: which single machine do we unplug first.

The insight this project rests on is that the data needed to catch these attacks already exists, in logs organisations already collect and already pay to store. The missing component is the layer that connects those records to each other. That is what we built, and it is why the system needs no new sensors, no new agents on endpoints, and no change to existing infrastructure.

3. Our solution

Resilience Graph AI is a working web application backed by two AI engines joined by a shared analysis pipeline. It accepts an authentication log, either one of the scenarios shipped with the product or a file uploaded by the user, and returns a complete incident investigation.

Capability	What the user gets	Status
Analyse any log	Select a scenario or upload a CSV. Every screen in the application re-renders on that data.	Live, per request
Campaign view	All 104 compromised accounts presented as one campaign rather than a single victim.	Live
Per account investigation	Open any account to get its own scoped incident, graph and report.	Live
Attack path graph	Click any machine to see every authentication involving it, and the blast radius across all attacker footholds.	Live
Single event scoring	Score one authentication event on demand using the real trained model.	Live
Next technique prediction	A ranked next move with a genuine transition probability.	Live
Threat group attribution	A ranked MITRE group with an auditable written justification.	Live, transparent retrieval
Threat Radar	External threat intelligence, India first, mapped to ATT&CK and cross referenced against the current incident.	Live feeds
Audit ready report	A printable incident report suitable for compliance records.	Live
Guided containment	Recommended response steps. Anything touching a critical asset requires human approval.	Simulated by design

The final row is deliberate. There is no live production network attached to this system, so no containment action is actually executed. Every such action is labelled as simulated in the interface. We consider stating this plainly to be part of the engineering, not a caveat to it.

4. How the system works, end to end

The analysis is a single function call that executes seven stages in order. Stages one and two are Engine 1, which finds the anomalies. Stages three to six are the shared spine, which turns those anomalies into an incident a human can act on. Stage seven is Engine 2, which looks forward and outward.

The analysis pipeline: seven stages, one function call

Every stage runs live on whatever log is submitted. Nothing is pre-computed at request time.

1	**Feature engineering** Turn each raw login record into seven behavioural measurements that describe how this account normally moves through the network. `src/engine1/lanl_detect.py :: engineer()`	**ENGINE 1** **Detection**
2	**Anomaly scoring** An IsolationForest trained only on normal behaviour scores every event from 0 to 100. Scores use fixed calibration anchors, so a score means the same thing on every upload. `models/iforest_lanl.joblib · api/cache/score_ref.json`	**ENGINE 1** **Detection**
3	**Correlation** Every event scoring 50 or above becomes an alert. Alerts are grouped into a single incident with a timeline, a severity and an ordered chain of techniques, instead of thousands of separate rows. `src/shared/correlate.py · ALERT_THRESHOLD = 50 · SESSION_GAP = 3600 s`	**SHARED SPINE**
4	**MITRE ATT&CK mapping** Each behaviour is matched to an official technique identifier. Names and descriptions are read from the parsed MITRE bundles, never generated, so an invented technique is structurally impossible. `src/shared/attack_mapper.py · 918 techniques`	**SHARED SPINE**
5	**Attack path graph** Alerts become edges in a directed graph of machine to machine movement. From it we compute total exposure, which valuable machines are reachable, and which single machine to isolate first. `src/shared/attack_graph.py · networkx · reachability from every pivot`	**SHARED SPINE**
6	**Gated response (SOAR)** Containment steps are drawn from the real MITRE mitigations for the observed techniques. Anything touching a critical asset requires human approval. All actions are simulated and labelled as such. `src/shared/soar.py`	**SHARED SPINE**
7	**Prediction and attribution** A first order Markov model ranks the attacker's likely next technique with a real transition probability. A transparent weighted retrieval ranks which known threat group the behaviour resembles, with a written justification. `src/engine2/build_predictor.py · src/engine2/attribution.py`	**ENGINE 2** **Predict and attribute**

One bundle out. All seven stages are a single function, `analyze_events()`, which returns one result object containing the payload for every screen in the application. The sample data shipped with the app is produced by calling the same function on a shipped log, so there is no separate demonstration path that could diverge from the real one.

Figure 1. The seven stage analysis pipeline. Every stage executes on each request.

5. System architecture

The application is one repository and deploys as one container. The frontend is a React single page application. The backend is FastAPI, which serves both the programming interface and, in production, the built frontend from the same origin. The analysis spine sits behind the live endpoints. Trained models and lookup tables are loaded once at start up.

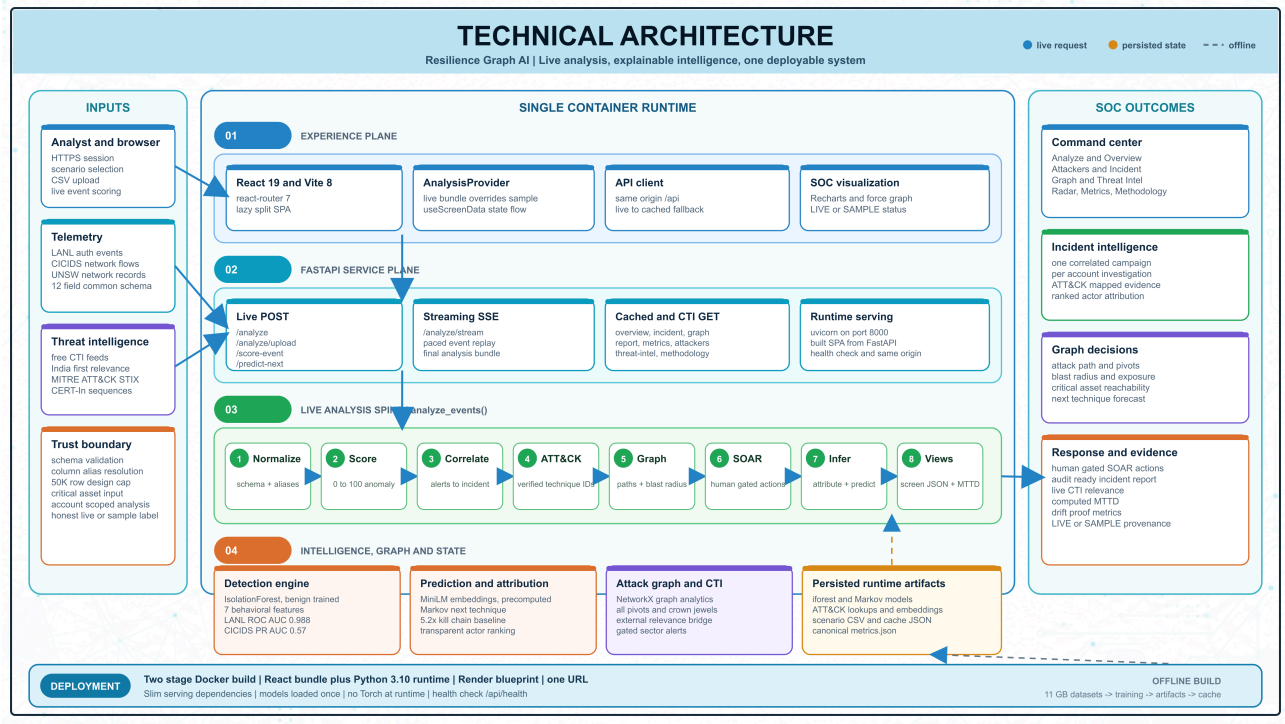


Figure 2. Technical architecture. Inputs, the single container runtime across four planes, and the outcomes delivered to a security team. A full resolution copy is in the repository at reports/technical_architecture_final.png.

5.1 The programming interface

Type	Endpoints	What happens
Live analysis	POST /api/analyze, POST /api/analyze/upload	Runs the complete seven stage pipeline on the submitted log and returns one bundle containing the payload for every screen.
Live model calls	POST /api/score-event, POST /api/predict-next	Single shot calls into the trained anomaly model and the transition model.
Live intelligence	POST /api/threat-radar	Fetches external threat intelligence and scores it against the current incident.
Streaming	GET /api/analyze/stream	Server sent events, replaying an incident event by event.
Cached reads	GET /api/overview, /incident, /graph, /report, /threat-intel, /attackers, /metrics	Serve a sample that is itself a real analysis of a shipped log, produced by calling the same pipeline offline.
Static	GET /	Serves the built frontend in production.

The last two rows matter for a specific reason. There is no separate mock data path in this system. The sample content that loads before a user runs their own analysis was generated by running the real pipeline over a real log and saving the result. A demonstration path that could quietly diverge from the real one does not exist, because it was never built.

5.2 Deployment topology

Environment	Topology
Local development	Two processes. The backend runs on port 8000, the frontend development server on port 5173 and proxies interface calls to it.
Production	One Docker container built in two stages. A Node stage builds the frontend, a slim Python stage serves the interface and the built frontend from a single origin, so no cross origin configuration is required. Hosted on Render.
Runtime requirements	No GPU. The deep learning framework is not installed in the deployed image at all. Sentence embeddings ship as a precomputed file.
Cold start from a clone	Trained models, ATT&CK lookups, embeddings, demonstration scenarios and the sample cache are all committed, so the application runs from a fresh clone with no dataset download.

6. Engine 1: behavioural detection

6.1 The idea

We never tell the model what an attack looks like. We show it a large volume of normal behaviour and ask a single question of each new event: how unusual is this. This is the only approach that can catch an attack nobody has catalogued yet, because it does not require the attack to have been seen before.

The difficulty is choosing what to measure. Raw log fields such as a user name or a machine name are useless to a model, because they are arbitrary identifiers. So we compute behavioural features that describe how a person moves through a network, and we compute them chronologically, per account.

6.2 The seven behavioural features

Feature	What it measures	Why an attacker trips it
`new_dst_for_user`	First time this account has ever logged into this machine	Attackers explore machines the real user never touches
`new_src_for_user`	First time this account logged in from this machine	The attacker operates from their own foothold machine
`user_distinct_dst_sofar`	How many different machines this account has reached so far	Rapid fan out is hunting behaviour, not daily work
`user_fail_rate_sofar`	Running share of failed logins for this account	Credential guessing leaves a trail of failures
`dst_rarity`	How rarely anyone at all logs into this destination	Attackers reach obscure but high value servers
`is_fail`	Whether this login failed	Basic corroborating signal
`is_ntlm`	Whether an older authentication protocol was used	Pass the hash attacks depend on it

These are not chosen by intuition alone. Measured on the real labelled data, the separation between benign and attacker behaviour is as follows.

Feature	Benign average	Attacker average	Separation
`new_dst_for_user`	0.0204	0.2821	13.8 times
`new_src_for_user`	0.0174	0.1054	6.1 times
`user_fail_rate_sofar`	0.0015	0.0081	5.4 times
`dst_rarity`	4.8715	9.8256	2.0 times
`is_ntlm`	0.0580	1.0000	Present in 100 percent of attack events

An attacker is 13.8 times more likely to touch a machine that the account they stole has never visited. That single behavioural fact carries most of the detection.

6.3 The model and its exact configuration

The shipped detector is an Isolation Forest. It isolates outliers by splitting the data at random. Anomalies are isolated in fewer splits because they sit apart from the crowd. It is fast, it requires no labels, and it behaves well in higher dimensions.

Setting	Value and reasoning
Estimators	200 trees
Maximum samples	4,096 per tree
Contamination	Set automatically
Random seed	42, fixed so results reproduce exactly
Training data	A random sample of 800,000 rows labelled benign. Attack rows are excluded from training entirely.
Normalisation	Standard scaling, fitted on the training sample only, so no information from the evaluation data reaches the model.
Score calibration	Raw scores are mapped to a 0 to 100 range using fixed anchor vectors stored in the repository, not the minimum and maximum of the current batch. A score therefore means the same thing across different uploads, and matches the single event endpoint exactly.
Use of labels	Evaluation only. Labels never enter training. This is what makes the reported detection score defensible rather than circular.

There is no look ahead leakage. Every running statistic, such as the count of distinct machines an account has reached, is computed using only that account's prior events, so the score of any given event never depends on the future.

7. The shared spine: turning alerts into a story

This is the part of the system that converts a pile of anomalies into something a human being can act on within minutes.

7.1 Correlation: 1,192 alerts become 1 incident

Any event scoring 50 or above becomes an alert. Rather than emitting 1,192 separate alerts, the system groups them into one incident carrying a timeline, a severity, the list of affected accounts, and an ordered chain of techniques that reads as a narrative. An hour of silence starts a new session. Severity is taken from the highest scoring event in the incident: 90 and above is critical, 75 and above is high, 50 and above is medium.

7.2 Mapping behaviour to the MITRE ATT&CK catalogue

MITRE ATT&CK is a free public catalogue that gives every known attacker technique an identifier, and it is the common vocabulary of the security industry. We infer the technique from observed behaviour, never from a text label.

Observed behaviour	Mapped technique	Plain meaning
The login failed	`T1110`	Brute force, guessing passwords
New machine reached using the older protocol	`T1550.002`	Pass the hash, reusing a stolen credential fingerprint
New machine reached	`T1021`	Remote services, legitimate remote login tools
Neither condition met	`No technique`	Treated as normal activity

Technique names, descriptions and recommended mitigations are read from the official MITRE data files, which we parse ourselves into a lookup table covering 918 techniques and 175 groups. The explanation text shown to the user is MITRE's own wording. No language model generates technique text anywhere in this system, which means a fabricated technique identifier is structurally impossible rather than merely unlikely.

7.3 The attack path graph

Every alert becomes an edge in a directed graph running from source machine to destination machine. From that graph we compute the four things a responder actually needs.

Question a responder asks	How the graph answers it	Result on the real campaign
Where is the attacker operating from	Machines with outbound attack movement	4 footholds. One machine, C17693, carries 670 of the 702 red team events.
How far can they reach	Reachable set from every foothold, combined	475 machines
Which valuable machines are exposed	Shortest path to each critical asset	18 critical assets reachable
What do we disconnect first	Ranking by betweenness centrality	Isolating C17693 alone severs 463 machines

We deliberately report two different numbers rather than one flattering one. Total exposure is 475 machines. What isolating a single choke point actually severs is 463. Presenting only the larger figure would overstate what one containment action achieves.

This distinction came out of a real defect we found and fixed. An earlier version assumed the attacker had one entry point, computed reachability from that machine alone, and consequently under reported exposure and wrongly cleared four critical assets as safe. The corrected version takes the union of reachable sets over every foothold, and a regression test now fails if anyone reintroduces the assumption.

On critical assets, an honest note. The public dataset is anonymised and carries no asset criticality labels at all. We therefore derive them from a stated heuristic: the machines that the largest number of distinct accounts authenticate to, which in practice surfaces domain controllers and authentication servers. On that basis the red team reached 13 of the estate's 20 most depended upon servers, including one that 17,808 accounts rely on. In a real deployment the operator supplies their own asset list instead. It is already an input parameter, not a value written into the code.

7.4 Guided response, with a human in the loop

Situation	Response mode
A critical asset is involved	Requires explicit human approval
High or critical severity	Simulated containment
Medium severity	Raise a ticket and enrich it
Low severity	Monitor only

Recommended containment steps are seeded from the real MITRE mitigations associated with the observed techniques, so the advice is MITRE's rather than ours. Every action is simulated, and labelled as such wherever it appears.

8. Engine 2: prediction and attribution

8.1 Predicting the attacker's next move

Given the techniques observed so far, what is likely to come next. We learned technique to technique transitions from 201 real attack sequences, made up of 145 MITRE group profiles and 56 campaigns, each containing at least six techniques, plus 4 analyst verified CERT-In advisories, giving 205 sequences in total.

Method	Top 3 accuracy	Outcome
Most frequent technique baseline	4.9%	Baseline
Kill chain order baseline	7.1%	Baseline built specifically to beat us
Recurrent neural network over sentence embeddings	27.2%	Lost. Published as a documented negative result.
First order Markov chain	36.5%	Shipped

Two things here matter more than the headline number.

The circularity trap, and how we escaped it. Our sequences are ordered using MITRE's kill chain tactic order, which runs from reconnaissance through to impact. A model could score well simply by re-learning that ordering, while learning nothing whatsoever about attacks. So we built a baseline whose entire strategy is to exploit that ordering, and we required our model to beat it. The Markov model beats it by a factor of 5.2, which is evidence that it is predicting genuine technique to technique transitions rather than re-deriving a sort order.

We shipped the model that won, not the impressive one. The neural network scored 27.2% and lost to a first order Markov chain at this data scale. We ship the Markov model and publish the neural result as a negative finding. At 205 sequences, a simple model is the correct engineering answer, and reporting the loss is more useful to a reader than hiding it.

The prediction returned by the interface is a genuine first order transition probability, computed as the observed count of a transition divided by the total from that state. An example from the shipped model is a transition observed at 52.5 percent. Where a state has never been seen, the system falls back to a frequency ranked list and explicitly scores those suggestions at zero, because there is no observed evidence for them. Data splits are made at the level of whole sequences, never inside one, which prevents leakage.

The non circular India test. The four CERT-In sequences are ordered by the real reported timeline rather than by our heuristic, and they appear only in the test set. The model scores 10.0% top 3 on them against 36.5% on the automatically ordered set. We treat that gap as a finding rather than a failure: real attacker orderings are harder, which demonstrates that part of the higher number was ordering driven. Both figures are published.

8.2 Attributing the activity to a known threat group

Given the observed techniques, which publicly documented threat group does this resemble. We score the observed technique set against 172 MITRE group profiles using a transparent weighted formula rather than a trained classifier.

```
score = 0.55 * coverage  +  0.20 * jaccard  +  0.25 * semantic_similarity

coverage             fraction of observed techniques in the group's public profile
jaccard              set overlap between observed and profile techniques
semantic_similarity  cosine distance between sentence embedding centroids
```

Every result carries a generated justification, for example that it matches three of four observed techniques, with a stated profile coverage and semantic similarity. A user can therefore audit the reasoning rather than trust a score.

The caveat we state before anyone asks. The built in evaluation, which hides 40 percent of a group's profile and then retrieves the group from the remainder, scores 100 percent at rank one. That number is close to trivial by construction, because it retrieves a public profile from a piece of itself. We never present it as a headline result. This component is transparent retrieval with a printed rationale, not a trained classifier, and on an authentication only log carrying three techniques the ranked group is context for an analyst, not a conclusion.

9. Threat Radar: external intelligence

A security team also needs to know what is happening outside its own walls. Threat Radar pulls free and legitimate threat intelligence feeds, maps each item to real ATT&CK identifiers, and cross references those against the incident currently loaded. Items relevant to India are ranked first, because the problem statement concerns Indian infrastructure. That covers mentions of India, of Indian agencies such as CERT-In and NCIIPC, of Indian sectors such as UPI, Aadhaar and the Reserve Bank, and of actors known to target India. At the time of writing that is 10 of 40 items.

The implementation uses only the Python standard library for network and document handling, so the deployed image gains no new dependencies. Each feed is isolated, meaning a dead source reports its own failure and never breaks the radar.

Technique mapping is deliberately precision first. It accepts explicit technique identifiers, then a curated table of about 70 phrases, then technique name matching. Reconnaissance and resource development techniques are excluded from name matching, because their official names are ordinary English nouns such as Software and Vulnerabilities, which matched innocent prose during testing. Every identifier is validated against the real catalogue before it is displayed.

Three honest notes about this feature.

- **CERT-In has no working feed.** Their address returns a success code together with an HTML page saying the address was not found, which is a disguised failure. We detected it, excluded the source, and added a guard that rejects HTML pretending to be a feed.
- **We do not scrape social media.** This was evaluated and rejected. It violates platform terms, it is actively blocked, and attributing activity to named individuals from public posts risks accusing the wrong people. We consider that an unacceptable failure mode for a security product.
- **No matches is a legitimate result.** Our demonstration incident is authentication based while the public news cycle is dominated by software vulnerabilities and malware, so the honest answer is often that there are no matches. The interface says exactly that, rather than manufacturing a connection to look impressive.

10. The application

Eight screens, all rendering the output of the analysis that is currently loaded.

Screen	What it shows
Analyze Log	Choose a shipped scenario or upload a CSV, then run the full pipeline.
Overview	Time to first alert, the active incident, and detector benchmarks.
Attackers	All 104 compromised accounts. Open any one for its own scoped incident.
Live Incident	Event by event replay, single event scoring, and the audit ready report.
Attack Graph	The interactive machine graph. Click any machine, or filter by account.
Threat Intel	ATT&CK mapping, ranked threat groups, and live next technique prediction.
Threat Radar	External intelligence cross referenced with the current incident.
Models and Metrics, Data and Methodology	The evidence tables, dataset descriptions and honesty notes.

The single most important element in the interface is the badge in the top bar, which reads either LIVE ANALYSIS or SAMPLE DATA. A viewer can always tell whether what they are looking at was computed moments ago from their own data, or is the shipped sample. We consider that badge a correctness feature rather than a decoration.

10.1 Robustness for real uploads

Two pieces of defensive engineering exist because of real failures, not speculation. The schema layer resolves column aliases case insensitively, so a column named username, account, src, source, dst or proto is understood without the user renaming anything. Timestamps are accepted either as epoch integers or as ISO-8601 date strings.

The second of those came from automated browser testing, which uploaded a file with ISO-8601 timestamps and found a crash, because every one of our own test files happened to use epoch integers. Real logs use dates. It is fixed, with a regression test that fails if it returns.

11. Data: what we used and why

Dataset	Size	Why it is in this project
LANL Cyber Security Events	11.2 million authentication events, 702 labelled red team events	The core asset. A real attack campaign inside a real network, with the attacker's own events labelled, which lets us prove detection rather than assert it.
CIC-IDS2017	2.3 million network flows	A network level benchmark containing distinct attack families, used to compare model families fairly.
UNSW-NB15	175,000 train and 82,000 test, official split	A second, independent benchmark, included to show the approach generalises beyond one dataset.
MITRE ATT&CK, Enterprise, ICS and Mobile	918 techniques, 175 groups	The technique vocabulary, the official mitigations, and the group profiles used for attribution.
CERT-In advisories	4 analyst verified sequences	Real Indian attack timelines, used as the non circular test set for prediction.

11.1 Why the LANL dataset matters most

It is a real 58 day capture from Los Alamos National Laboratory that includes an actual red team exercise, and critically, the red team's own events are labelled. Most public security datasets are either synthetic or unlabelled. This one lets us state a detection figure that can be checked.

The raw authentication file is roughly 70 gigabytes uncompressed. Our preparation script streams the compressed file rather than expanding it, reading 519 million lines, keeping every event involving a compromised account plus a one in four hundred background sample, joining the red team labels on the combination of time, source account, source machine and destination machine, and stopping early once past the attack window. The

result is 11.2 million rows containing 702 malicious events. We note openly that 98.2 percent of the 715 red team records have an exact authentication counterpart, a known quirk of the dataset that we state rather than quietly round away.

CIC-IDS2017 is split by day, training on Monday to Wednesday benign traffic only and testing on Thursday and Friday, which contain seven attack families the model never saw. The destination port column is dropped so the model cannot simply memorise identifiers. Both choices exist to prevent the model scoring well for the wrong reason.

11.2 Why we added the Mobile catalogue mid project

A teammate's verified CERT-In advisory described an Android banking trojan whose techniques did not exist in the Enterprise and ICS catalogues. Every one of those identifiers would have been silently discarded, and the sequence would have been quietly wrong. Adding the Mobile matrix took the catalogue from 794 to 918 techniques. This matters for the problem statement, because India's threat landscape is heavily mobile.

12. Technology choices, and what we rejected

Every significant choice below was made against a named alternative. Where the alternative won on a particular benchmark, we say so.

12.1 Modelling choices

Decision	What we chose	What we rejected, and why
Detection approach	Unsupervised anomaly detection trained only on benign data	Supervised classification. It needs labelled attacks, which real organisations do not have, and it can only recognise attack types present in its training set. Our whole problem is the attack nobody has catalogued.
Detection model	Isolation Forest	One class support vector machines, which scale poorly to millions of rows. A deep autoencoder was also built and did beat Isolation Forest on the network dataset, at 0.570 against 0.473 precision recall area. We report that openly, and still ship Isolation Forest for the authentication task, where it performs strongly and needs no GPU.
Prediction model	First order Markov chain	A recurrent neural network over sentence embeddings, which scored 27.2% against the Markov model's 36.5%. At 205 sequences the simpler model is genuinely better. We shipped the winner and published the loss.
Attribution method	Transparent weighted retrieval with a printed rationale	A trained classifier. With 172 groups and few labelled campaigns it would overfit, and it could not explain itself. An analyst must be able to audit an attribution claim.
Technique descriptions	Read verbatim from the official MITRE data files	Generating technique text with a language model. That introduces the possibility of a confidently invented technique identifier, which in a security tool is a serious defect, not a cosmetic one.
Evaluation metric	Precision recall area under curve, and true positive rate at a fixed false positive rate	Accuracy. At an attack prevalence of 0.006 percent, a model that answers benign every single time scores 99.994 percent accuracy while catching nothing at all.

12.2 Engineering choices

Decision	What we chose	What we rejected, and why
Graph analytics	networkx, in memory	A dedicated graph database such as Neo4j. It would add an entire service to operate for no benefit at our data scale, where analysis completes in seconds. We state the ceiling of roughly 50,000 events per analysis and name the graph database as the documented next step rather than pretending the limit does not exist.
Backend framework	FastAPI with uvicorn	Django, which brings an ORM and admin layer we have no use for. Flask, which would need extra parts for streaming and request validation that FastAPI provides directly.
Frontend	React with Vite	Next.js. Server side rendering earns nothing for an analyst tool behind a login, and it would complicate packaging the whole product as one container.
Threat intelligence client	Python standard library for network and document parsing	Third party request and feed parsing libraries. Avoiding them keeps the deployed image at zero additional dependencies, which matters on a constrained free hosting tier.
Sentence embeddings	A pretrained compact sentence transformer, precomputed at build time and shipped as a file	Calling a hosted embedding service at runtime, which would add cost, latency, an external dependency and a privacy question. Our approach means the deep learning framework is not installed in the production image at all.
Deployment	One Docker container on a free hosting tier	A multi service architecture. For a single analyst tool it would add operational burden and cost with no user visible benefit. One container means one address and a reviewer can run it in minutes.
Log ingestion	CSV upload and shipped scenarios	A direct live connector to a security platform. We have no such platform to connect to, and building an untestable integration would be theatre. Connectors are the first item on the roadmap.
External data collection	Free, legitimate, published threat feeds	Scraping social media platforms. It breaches platform terms, it is actively blocked, and person level attribution from public posts risks naming innocent people.

13. Results in full

13.1 Detection

Dataset	Metric	Result
LANL	ROC area under curve	0.988
LANL	True positive rate at 5 percent false positive rate	96.9% (680 of 702 attacks caught)
LANL	True positive rate at 1 percent false positive rate	51.4%
LANL	Behavioural only ROC, protocol signal removed	0.929
CIC-IDS2017	Precision recall area, autoencoder	0.570
CIC-IDS2017	Precision recall area, Isolation Forest	0.473 (3.1 times random, 4.8 times the rule baseline)
CIC-IDS2017	Precision recall area, random baseline	0.155
CIC-IDS2017	Precision recall area, naive rule baseline	0.098 (worse than random)
UNSW-NB15	ROC area under curve	0.829
UNSW-NB15	Precision recall area	0.867

The protocol ablation, and why we ran it. Every red team login in the dataset used the older NTLM protocol, against about 6 percent of benign logins. That is an extremely powerful signal, and also a brittle one: it is specific to this dataset and an attacker could simply switch protocols to evade it. So we deleted the feature entirely and re-ran the evaluation. Detection held at 0.929. The result is driven by generalisable behaviour rather than one fragile artifact. It would have been easy to keep the higher number quietly.

The rule baseline is worse than random, and we published it. A naive high volume rule scores 0.098 against a random floor of 0.155, because stealthy attacks are low volume by definition. We report it because it is precisely why simple threshold rules fail against this class of attack.

13.2 Prediction and attribution

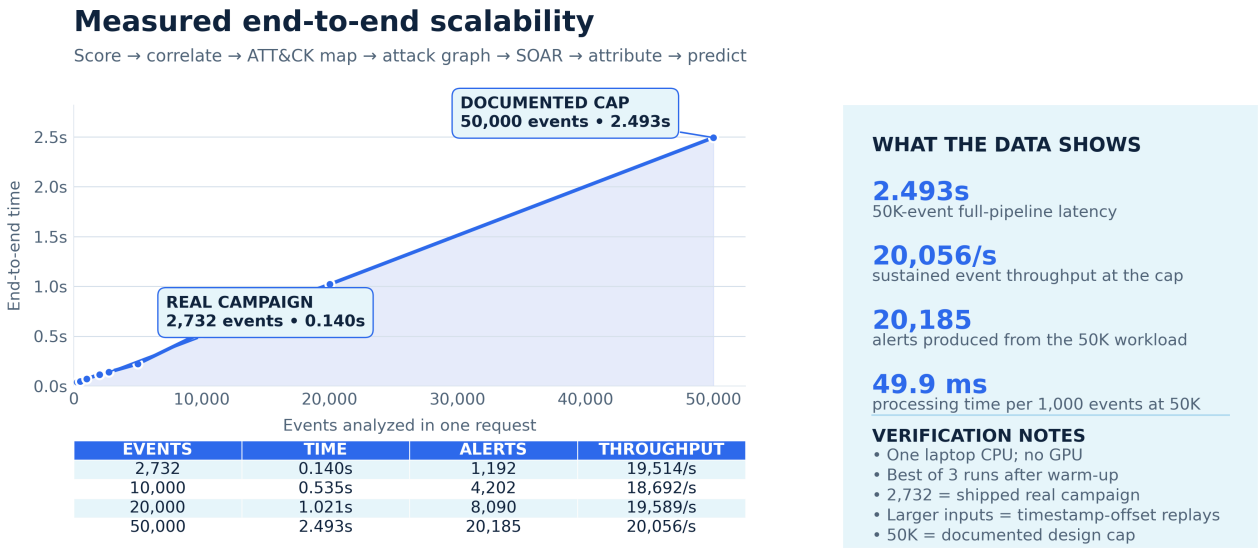
Metric	Result
Markov top 3 accuracy, automatically ordered sequences	36.5%
Anti circularity margin over the kill chain baseline	5.2 times
Neural network top 3 accuracy, documented negative result	27.2%
CERT-In verified sequences, top 3 accuracy	10.0%
Technique embeddings, same tactic cosine similarity	0.412 against 0.327 for random pairs
MITRE group profiles used for attribution	172

13.3 Operational output on the live campaign

Output	Value
Events analysed, alerts raised, incidents produced	2,732, then 1,192, then 1
Compromised accounts identified	104
Attack graph size	479 machines, 502 movements, 4 attacker footholds
Critical assets reachable by the attacker	18
Total exposure	475 machines
Effect of isolating the single best choke point	463 machines severed
Events concentrated on the primary foothold	670 of 702 red team events on C17693

14. Performance and scalability

Rather than assert that the system scales, we measured it. The complete pipeline was timed at nine input sizes on an ordinary laptop processor with no GPU.



EVENTS	TIME	ALERTS	THROUGHPUT
2,732	0.140s	1,192	19,514/s
10,000	0.535s	4,202	18,692/s
20,000	1.021s	8,090	19,589/s
50,000	2.493s	20,185	20,056/s

Figure 3. Measured end to end analysis time against input size, with the measurement table and verification notes.

The shipped demonstration campaign completes in 0.140 seconds. The documented upper limit of 50,000 events in a single analysis completes in 2.493 seconds. Cost per event stays effectively flat to 20,000 events and rises modestly at the cap, so we describe the result as fast rather than claiming perfect linearity. Measurements are the best of three runs after a warm up call, because the first call loads the model and would otherwise be measuring start up rather than analysis. Runs above 2,732 events replay the real campaign with offset timestamps, which is disclosed on the chart itself.

15. How we kept ourselves honest

Four rules were set at the start of the project and held to, including on the occasions when they cost us a better looking number.

- **Never report accuracy.** At this class imbalance, accuracy is a meaningless statistic that flatters a useless model. We report precision recall area and true positive rate at a fixed false positive rate, which is what an analyst actually experiences.
- **Always show baseline lift.** Random and rule baselines were built before the real model, so we could not tune toward a flattering comparison after the fact. When the rule baseline came out worse than random, we published that too.
- **Build the baseline designed to defeat you.** The kill chain order baseline exists purely to test whether our own predictor was cheating. We report the margin over it, and when the neural model lost to the simple one, we shipped the simple one.
- **Nothing fabricated on screen.** Every displayed number traces to the current analysis or to a labelled citation.

The fourth rule cost us work, because it meant removing things we had already built. We list them because a reviewer deserves to know what a team removed, not only what it added.

What we removed	Why it had to go
Invented trend lines in the interface	The arrays behind them were decorative fiction. Replaced with real anomaly scores.
A hard coded detection time claim	It was a fixed string. It is now measured from the timestamps of the log being analysed, with the industry comparison shown as a citation.
A fabricated critical asset	The code selected the middle element of a list and presented it as a finding. Replaced with a stated, defensible heuristic.
A stale anti circularity figure	The interface claimed a margin our own report contradicted. We found it in a self audit, corrected it, and then fixed the underlying cause.

That last item produced a structural change rather than a patch. Evaluation scripts now write a single metrics file, which the interface reads directly. Hand copied numbers can no longer go stale, because numbers are no longer hand copied.

16. Testing and engineering

Layer	Coverage
Automated tests	29 tests covering pipeline correctness, campaign versus per account scoping, multi foothold graph reachability, cross screen consistency of critical assets, intelligence mapping precision, and resilience to a dead feed.
Browser end to end	An automated agent drove a real browser through 15 user flows. 14 passed.
Deployment	The container build is verified, and the running container is smoke tested including a live external intelligence fetch from inside it.

The browser testing earned its place. It uploaded a file with ISO-8601 timestamps and found a crash, because every fixture we had written happened to use epoch integers, and real logs do not. That is a defect a judge would have triggered during a demonstration. It is fixed with a regression test.

Several tests exist specifically to prevent defects we caused once already: the vulnerability feed is ordered by vendor rather than by date and must be sorted explicitly, a ransomware campaign flag must never be mapped as the ransomware technique itself, and a graph edge must remember every account that used a machine pair rather than only the first.

17. Limitations we state before you find them

Limitation	Our position
The demonstration incident carries only three ATT&CK techniques	The dataset is authentication logs only, with no process, file or network telemetry. Authentication behaviour can honestly evidence pass the hash, brute force and remote services, and nothing more. We refuse to invent techniques the data cannot support. Richer telemetry deepens the chain automatically.
The attribution evaluation scores 100 percent	It is close to trivial by construction, since it retrieves a public profile from part of itself. We never headline it. The component is transparent retrieval, not a classifier.
Prediction on the CERT-In sequences is only 10 percent	That is the honest, non circular figure, and the gap against the automatically ordered set is itself the finding. Prediction is a supporting feature. The case rests on detection and correlation.
Critical assets are a heuristic	The dataset carries no criticality labels. We state the heuristic openly. A real deployment supplies an asset inventory, which is already an input parameter.
Containment is simulated	There is no live network to act upon. Every action is labelled simulated and requires human approval.
The India scenarios are synthetic	The AIIMS and CBSE scenarios are generated logs styled after real reported incidents, and are labelled as synthetic in the interface. The LANL campaign is the real data.
Graph analytics are held in memory	Comfortable to roughly 50,000 events per analysis, as measured. Beyond that the approach is to shard by tenant or time window, or move to a graph database.
The application has no user authentication	It is a single analyst demonstration and the login screen is a splash by design. We make no security claim about the application itself.

18. Impact, deployment path and roadmap

Dimension	Situation today	With Resilience Graph AI
Detection	About 10 days median dwell time	First correlated alert within the log window
Analyst load	1,192 alerts to triage	1 incident carrying a narrative
Containment	Which of 479 machines do we isolate	Isolate 1 machine, sever 463 machines of exposure
Attribution	Manual reading of threat intelligence	A ranked group with an auditable justification
External intelligence	A separate portal, read separately	Cross referenced against your own techniques

No new sensors are required. The system makes the logs an organisation already collects tell the story that is already in them. That is what makes it deployable at the organisations that need it most: hospitals, examination boards, grid operators, and the large share of government entities running end of life infrastructure who are least able to fund a 24 hour security operations centre.

Horizon	Work
30 days	Connectors for the common log platforms, so logs arrive without manual upload.
90 days	Operational technology and industrial control coverage, and a graph database backend for larger estates.
6 months	A sector level view for a national response team, and real containment actions placed behind formal change control.

19. Reproducing this work

The application runs from a fresh clone with no dataset download, because the trained models, catalogue lookups, embeddings and demonstration scenarios are all committed to the repository.

```
# run the application
pip install -r requirements-deploy.txt
python -m uvicorn api.main:app --port 8000
cd frontend && npm install && npm run dev

# retrain everything from the raw datasets
pip install -r requirements.txt
python -m src.engine1.prep_lanl    && python -m src.engine1.lanl_detect
python -m src.engine1.prep_cicids  && python -m src.engine1.anomaly
python -m src.shared.parse_attack
python -m src.engine2.build_embeddings
python -m src.engine2.build_sequences && python -m src.engine2.build_predictor
python -m scripts.build_cache
```

Every evaluation script writes a report to the reports directory. Those reports are the audit trail behind every number in this document.

20. Glossary

Term	Meaning
ATT&CK	MITRE's public catalogue of attacker techniques, each with an identifier such as T1110.
Blast radius	Everything an attacker can reach from where they currently are.
CERT-In	India's national Computer Emergency Response Team.
Choke point	A machine that many attack paths pass through, and therefore the best one to isolate.
Critical asset, or crown jewel	A machine worth protecting most, such as a patient database or a domain controller.
Dwell time	How long an attacker remains undetected inside a network.
False positive rate	The share of normal events wrongly flagged. This is what causes alert fatigue.
Isolation Forest	An algorithm that finds outliers by measuring how easily a point can be separated from the rest.
Lateral movement	Moving from machine to machine inside a network after gaining entry.
NTLM	An older Windows authentication protocol, vulnerable to pass the hash.
Pass the hash	Logging in using a stolen credential fingerprint instead of the password itself.
PR-AUC	Precision recall area under curve. The correct substitute for accuracy when the event of interest is rare.
Red team	Authorised attackers who simulate a real intrusion. Their recorded actions are the ground truth labels.
ROC-AUC	The probability that the model ranks a random attack above a random benign event. 1.0 is perfect, 0.5 is a coin toss.
SIEM	The log collection platform a security team runs on.
SOAR	Security orchestration, automation and response. The layer that acts after detection.
SOC	Security Operations Centre. The team watching for attacks.
TPR	True positive rate, also called recall. The share of real attacks caught.
Unsupervised learning	Training without labels. The model learns what normal looks like and flags deviation from it.